AutoSecAudit 2.0 — Viva Defense Master Guide

Comprehensive Technical Reference: Complete Full Forms, Security Definitions, Architecture Concepts & Viva Defense Explanations

Project: AutoSecAudit (Phase 3 Defense)	**Institution: Don Bosco Institute of Technology (Autonomous under VTU)**
Department: Computer Science & Engineering	**Faculty Guide: Ms. Ranjitha R (Assistant Professor, Dept of CSE)**
Team: Kodi Vishruth Aithal, Thilak N, Vinaya Kumar B, Vishal Gowda B	**Benchmark Baseline: OWASP Juice Shop | 14 Plugins | k=8 Workers**

Part 1: Alphabetical Quick-Reference of All Full Forms

This table compiles every abbreviation found across all 21 slides, system diagrams, and research benchmarks.

Acronym	Full Form	Operational Meaning in AutoSecAudit
API	Application Programming Interface	Standard contract allowing two programs to communicate and exchange structured data.
BFS	Breadth-First Search	Graph traversal algorithm used by our dynamic web crawler to discover endpoints level-by-level (d_max=3).
CI/CD	Continuous Integration / Continuous Deployment	Automated pipelines that build, test, audit, and deploy code commits automatically.
CLI	Command-Line Interface	Terminal execution tool (`python cli.py`) providing `--fail-on` gating and headless scan modes.
CmdInj	Operating System Command Injection	High-severity vulnerability allowing attackers to execute raw server shell commands (CWE-78).
CORS	Cross-Origin Resource Sharing	W3C standard allowing servers to permit or block browser requests from external origins (CWE-942).
CSE	Computer Science & Engineering	Academic department under which this major project is submitted.
CSRF	Cross-Site Request Forgery	Attack tricking authenticated victims into submitting unauthorized web requests on trusted sites.
CVE	Common Vulnerabilities and Exposures	Standard public identification catalog for published software vulnerabilities (e.g., CVE-2023-38606).
CVSS	Common Vulnerability Scoring System	Standard numerical metric (0.0 to 10.0) measuring technical exploit severity (NIST NVD v3.1).
CWE	Common Weakness Enumeration	Universal dictionary of software vulnerability flaw types maintained by MITRE (e.g., CWE-89).
DAST	Dynamic Application Security Testing	Black-box security testing evaluating running web applications by transmitting live HTTP attack payloads.
DBIT	Don Bosco Institute of Technology	Autonomous engineering institution accredited by NAAC with 'A' Grade.
DevSecOps	Development, Security, and Operations	Engineering discipline embedding automated security testing directly into continuous software delivery.
DOM	Document Object Model	Browser programmatic tree structure representing HTML tags; vulnerable in DOM-based XSS attacks.
ETag	Entity Tag	HTTP response header used for web cache validation; leveraged by our Baseline Response Cache.
FN	False Negative	Security flaw: a real vulnerability exists in the target but was completely missed by the scanner.
FP	False Positive	Noise flaw: safe, harmless code is erroneously reported as a security vulnerability.
HMAC	Hash-based Message Authentication Code	Cryptographic authentication mechanism combining a secret key with cryptographic hash functions.
HTML	HyperText Markup Language	Standard markup language used to structure web pages, forms, input tags, and links.
HTTP / HTTPS	Hypertext Transfer Protocol / Secure	Foundational application protocol for transferring hypermedia and secure encrypted web communications.
IDOR	Insecure Direct Object References	Access-control failure where changing a parameter (e.g. `user_id=45`) exposes another user's record (CWE-639).
IEEE	Institute of Electrical and Electronics Engineers	Global standards body; standard formatting used for our accompanying research paper.
JSON	JavaScript Object Notation	Lightweight, text-based data interchange format used in REST APIs and OpenAPI schemas.
JWT	JSON Web Token	Compact URL-safe authentication token; tested for signature evasion (`alg: none`) and weak HMAC secrets.
NAAC	National Assessment and Accreditation Council	Indian higher education quality assurance body (DBIT is accredited with 'A' Grade).
NBA	National Board of Accreditation	Tier-1 engineering academic accreditation authority in India.
NIST	National Institute of Standards and Technology	US agency responsible for federal security standards, the NVD database, and CVSS scoring.
NVD	National Vulnerability Database	US government REST API repository queried dynamically by our tool for live threat scoring.

OpenAPI / Swagger	OpenAPI Specification (formerly Swagger)	Standardized, machine-readable format (JSON/YAML) defining backend REST routes, parameters, and headers.
OWASP	Open Worldwide Application Security Project	Global nonprofit foundation defining software security standards (e.g. OWASP Top 10: 2021).
PathTrav	Path Traversal / Directory Traversal	Exploit manipulating file paths (`../../etc/passwd`) to access restricted server files (CWE-22).
PBKDF2	Password-Based Key Derivation Function 2	Cryptographic key-stretching algorithm used for secure salted password hashing in `data/users.db`.
PCI-DSS	Payment Card Industry Data Security Standard	International technical mandate required for entities handling credit card transactions.
PR	Pull Request	Developer code submission in Git; evaluated and gated automatically by our `--fail-on` CI/CD rule.
REST	Representational State Transfer	Architectural model using standard HTTP verbs (GET, POST, PUT, DELETE) for stateless APIs.
SAST	Static Application Security Testing	White-box security testing analyzing dormant source code without compiling or running the system.
SHA-256	Secure Hash Algorithm 256-bit	Cryptographic hash function used in our Delta Regression engine to generate finding fingerprints.
SQLi	Structured Query Language Injection	High-impact flaw where unescaped SQL syntax is injected into backend database queries (CWE-89).
SSE	Server-Sent Events	Unidirectional HTTP server push technology used to stream live 7-stage scan progress to our web UI.
SSRF	Server-Side Request Forgery	Exploit forcing the backend server to make unauthorized requests to internal networks or cloud metadata (CWE-918).
SSTI	Server-Side Template Injection	Vulnerability allowing attackers to execute arbitrary server code via template syntax (e.g. `{{7*7}}`) (CWE-1336).
TN	True Negative	Correct assessment: safe, non-vulnerable code is correctly recognized and ignored by the scanner.
TP	True Positive	Correct assessment: a genuine, exploitable security flaw is successfully discovered and reported.
UI	User Interface	Visual frontend dashboard built with Flask, Tailwind, and Jinja2 for interacting with the audit engine.
URL / URI	Uniform Resource Locator / Identifier	Global address reference used to specify web application resources and target hostnames.
USN	University Seat Number	Unique university registration identifier assigned to each engineering student under VTU.
VTU	Visvesvaraya Technological University	State technical university in Karnataka under which DBIT is affiliated and autonomous.
XSS	Cross-Site Scripting	Client-side code injection flaw enabling attackers to execute malicious JavaScript in victims' browsers (CWE-79).
YAML	YAML Ain't Markup Language	Human-friendly data serialization standard commonly used in OpenAPI specifications and CI/CD configs.

Part 2: Categorized Viva Defense Deep Dives

Detailed technical concepts, architectural mechanisms, and defense answers likely to be examined by the review panel.

A. Security Taxonomies & Threat Standards

• **OWASP Top 10 (2021):** The global benchmark of web application security risks. AutoSecAudit maps every finding into its corresponding OWASP category: **A01 (Broken Access Control)** for IDOR and auth flaws; **A03 (Injection)** for SQLi, XSS, and CmdInj; **A05 (Security Misconfiguration)** for missing defensive headers, default accounts, and directory listings; and **A07 (Identification & Authentication Failures)** for JWT bypasses.

• **NIST NVD REST API v2.0:** The US National Vulnerability Database. When AutoSecAudit detects exposed server banners (e.g., Apache 2.4.49, Werkzeug 2.0), it queries the live NIST NVD REST API to fetch verified **CVE IDs**, official **CVSS v3.1 base scores**, and published exploit references, avoiding subjective severity guesswork.

• **CVSS v3.1 Scoring Formula:** Common Vulnerability Scoring System. Generates scores from 0.0 to 10.0 using vector strings covering Attack Vector (AV), Attack Complexity (AC), Privileges Required (PR), User Interaction (UI), Scope (S), Confidentiality (C), Integrity (I), and Availability (A). Severities are classified as: **Low (0.1–3.9)**, **Medium (4.0–6.9)**, **High (7.0–8.9)**, and **Critical (9.0–10.0)**.

• **CWE Taxonomy (MITRE):** Common Weakness Enumeration provides common language for underlying software architecture errors. Examples: **CWE-89** (SQL Injection), **CWE-79** (XSS), **CWE-918** (SSRF), **CWE-1336** (SSTI), **CWE-78** (OS Command Injection), and **CWE-639** (Authorization Bypass through User-Controlled Key / IDOR).

• **DAST vs. SAST: DAST (Dynamic Application Security Testing)** is black-box evaluation that audits a running application over HTTP/HTTPS by injecting active attack payloads—this is what AutoSecAudit does. **SAST (Static Application Security Testing)** scans dormant source code lines without running the software. DAST discovers runtime, deployment, and network misconfigurations that SAST cannot detect.

B. Audited Vulnerabilities & Exploit Mechanisms

• **SQL Injection (SQLi) — CWE-89:** Occurs when untrusted user input is concatenated directly into database queries. AutoSecAudit verifies three variations: **(1) Error-Based:** induces database syntax errors; **(2) Boolean Blind:** checks differential responses for `AND 1=1` vs `AND 1=2`; **(3) Time-Based Blind:** uses `pg_sleep()` or `WAITFOR DELAY` and measures network latency. *Remediation:* Parameterized queries (Prepared Statements).

• **Cross-Site Scripting (XSS) — CWE-79:** Occurs when user-supplied input is echoed in the browser without HTML encoding. Evaluates **Reflected XSS** (payload returned immediately in HTTP response) and **DOM-based XSS** (payload executed via JavaScript sinks like `innerHTML` or `document.write`). *Remediation:* Context-aware HTML entity encoding and strict Content Security Policy (CSP).

• **Cross-Origin Resource Sharing (CORS) Misconfiguration — CWE-942:** Occurs when the server echoes back `Access-Control-Allow-Origin: *` or dynamically reflects untrusted origins while setting `Access-Control-Allow-Credentials: true`. This allows malicious third-party websites to steal sensitive authenticated session data via cross-origin JavaScript fetch calls.

• **Server-Side Request Forgery (SSRF) — CWE-918:** Tricks the backend web server into initiating unauthorized network requests to internal, un-routable private IPs. AutoSecAudit injects probes targeting loopback addresses (`127.0.0.1`, `localhost`) and cloud metadata endpoints (`http://169.254.169.254/latest/meta-data/`) to verify if cloud IAM credentials can be leaked.

• **Server-Side Template Injection (SSTI) — CWE-1336:** Occurs when template engines (Jinja2, Twig, Mako) evaluate user input directly as code. AutoSecAudit injects mathematical expressions like `{{7*7}}` or `${{7*7}}`. If the server computes and renders 49, Remote Code Execution (RCE) is confirmed.

• **JWT Signature Bypass (alg: none) — CWE-347:** JSON Web Tokens contain three Base64URL parts: Header, Payload, and Signature. Vulnerable backends accept tokens where the header declares `alg: none` without verifying the signature, allowing attackers to forge administrative tokens with full privileges.

• **Insecure Direct Object References (IDOR) — CWE-639:** A broken access control flaw where internal database record IDs are exposed in URLs (e.g. `/api/invoices/1042`). AutoSecAudit modifies object IDs across user sessions to verify if users can access other accounts without authorization.

C. Web Discovery & Reconnaissance

• **Dynamic BFS Web Crawler:** Implements **Breadth-First Search** graph traversal using Python's `deque` FIFO queue. Discovers all reachable application endpoints level-by-level up to a maximum depth limit of 3 (`d_max = 3`). Automatically parses HTML `<form>` tags, HTTP method types (GET/POST), input parameter names, and URL query strings, populating the unified scanner target pool.

• **OpenAPI 3.0 / Swagger Schema Ingestion:** Modern REST applications (React/Angular frontends) often lack traditional HTML hyperlinks. AutoSecAudit natively ingests OpenAPI/Swagger schema files in JSON or YAML, extracting backend route paths, path variables, HTTP verbs (GET, POST, PUT, DELETE), and required authentication headers.

• **Server-Sent Events (SSE):** A lightweight, unidirectional HTTP streaming protocol (`text/event-stream`). Unlike bi-directional WebSockets which require custom protocol handshakes, SSE operates over standard HTTP/HTTPS. Used by AutoSecAudit to stream real-time progress across 7 distinct execution stages (5% to 100%) to the web dashboard without browser reloads.

D. Concurrency Engine, Intelligence & Delta Auditing

• **ThreadPool Concurrency (k=8 Workers):** Python's `concurrent.futures.ThreadPoolExecutor` orchestrates 14 independent scanner plugins concurrently across 8 parallel worker threads. Slashes end-to-end audit duration from **184.2 seconds (~3.1 min) down to 64.8 seconds (~1.0 min)**, delivering a **2.84x speedup** and a **64.8% latency reduction**.

• **Baseline HTTP Response Cache:** An in-memory cache that captures initial baseline responses (status codes, headers, and body hashes) from the target server. When 14 plugins probe the same endpoints, the engine resolves baseline comparisons locally, preventing server rate-limiting and eliminating redundant network traffic.

• **Compound Finding Deduplication Algorithm:** When multiple tools flag the same endpoint, reporting duplicate alerts causes developer fatigue. AutoSecAudit aggregates raw findings using a 4-part compound tuple:
K = <host, port, endpoint, weakness_type>
On the OWASP Juice Shop benchmark, this algorithm consolidated **76 raw alerts down to 47 unique actionable findings**, delivering a **-38.2% duplicate noise reduction** with **97.9% precision**.

• **Set-Theoretic Delta Regression Auditing:** Traditional security scanners lack memory between releases. AutoSecAudit computes a deterministic SHA-256 fingerprint H(f) for every finding identity, partitioning findings between current scan S_t and baseline S_(t-1) into three disjoint sets:
  • **Delta_NEW = { f in S_t | H(f) not in H(S_(t-1)) }**: Newly introduced security flaws (Red alert badge).
  • **Delta_FIXED = { f in S_(t-1) | H(f) not in H(S_t) }**: Successfully resolved and patched vulnerabilities (Green badge).
  • **Delta_UNCHANGED = { f in S_t | H(f) in H(S_(t-1)) }**: Persistent technical debt carried forward (Neutral gray).

• **CI/CD Build Policy Gating (--fail-on):** The CLI runner enforces automated build policies by setting process exit codes:
ExitStatus = 1 (if any finding has Severity ≥ threshold θ, e.g. Critical) → Blocks Git Pull Request.
ExitStatus = 0 (otherwise) → Passes CI/CD pipeline and approves deployment.

E. Evaluation Formulas, Benchmarks & Metrics

• **Precision Formula:** Precision measures the proportion of reported security alerts that were genuine vulnerabilities:
Precision = TP / (TP + FP) = 46 / (46 + 1) = 97.87% ≈ 97.9% (TP = 46 verified True Positives, FP = 1 False Positive).

• **Noise Reduction Formula:** Quantifies the suppression of redundant alerts through compound deduplication:
Noise Reduction = (Raw Alerts - Correlated Findings) / Raw Alerts = (76 - 47) / 76 = 38.16% ≈ 38.2%

• **Speedup Ratio Formula (S):** Quantifies the concurrency acceleration achieved by our 8-worker threadpool:
Speedup = T_sequential / T_parallel = 184.2 s / 64.8 s = 2.84x (a 64.8% reduction in audit duration).

• **Automated Test Suite Coverage:** AutoSecAudit features an **80-test benchmark verification test suite** (and 124 passing tests across the entire codebase) covering the crawler, OpenAPI parser, 14 plugins, deduplication engine, delta diffing, and CI/CD gating, running to **100% PASS in 13.68 seconds**.

F. Panel Rapid-Fire Defense Cheat Sheet (Top Examiner Questions)

Q1: Why DAST instead of SAST?	SAST only inspects static source code and misses runtime deployment misconfigurations, web server headers, and network interactions. DAST audits live running applications over HTTP/HTTPS, proving actual exploitability.
Q2: What is your unique research contribution?	Three core innovations: (1) Compound 4-tuple Deduplication (-38.2% noise reduction, 97.9% precision), (2) Set-Theoretic Delta regression tracking fixed vs new flaws between builds, and (3) Automated CI/CD `--fail-on` build gating.
Q3: Why ThreadPool instead of Asyncio?	Security scanner probes make heavy blocking I/O socket calls via Python's standard `requests` library. ThreadPoolExecutor (k=8) achieved a 2.84x speedup without breaking compatibility with synchronous security libraries.
Q4: How does Delta diffing track patched bugs?	By hashing finding identities into deterministic SHA-256 keys. If a hash exists in baseline scan S_(t-1) but is absent in current scan S_t, the engine mathematically confirms that the flaw is Delta_FIXED.